



Data Mining

Unsupervised ML: Association Rules, Clustering

Eric Atwell, School of Computing,
University of Leeds

(including re-use of teaching resources from other sources, esp.
Knowledge Management by Stuart Roberts,
School of Computing, University of Leeds)

Summary

- compare classification with **association rules**.
- an algorithm for mining association rules.
- compare classification with **clustering**.
- algorithms for clustering
- the difference between Supervised ML (classifiers) and Unsupervised ML (association, clustering)

Classifiers v Association Rules

- ML classifiers predict the class attribute/feature.
- ML classifier needs training data with each instance labelled with its class or “answer” : supervised ML.
- But what if training data is not labelled with class? – unsupervised ML
- We may look for rule-based patterns involving *any* attributes: association rules
- From data on shopping baskets:
“shoppers who buy nappies often buy wine as well”
does not predict the “class” of shoppers

Example: PCs in a lab

- For each PC, IT expert assesses Risk of virus infection: High/Medium/Low
- For each PC, record features which may predict risk:
- F: connected to shared File-server
- S: connected to shared Scanner
- I: infected previously
- ML classifier: from features F,S,I predict class Risk
- ML Association: find any rules linking F,S,I,Risk

Learning Association Rules

- The search space for association rules is much larger than for classifier decision trees (eg J48 in Weka).
- To reduce the search space we consider only rules with large ‘coverage’ (lots of instances match lhs).
- The basic algorithm is:
 - Generate all candidate rules with coverage greater than some agreed minimum coverage;
 - Select only those rules with accuracy greater than some agreed minimum accuracy (eg 100%!).

Rule Generation

- First find all combinations of attribute-value pairs with a pre-specified minimum coverage.
- These are called *item-sets*.
- Next Generate all possible rules from the item sets;
- Compute the coverage and accuracy of each rule.
- Prune away rules with accuracy below pre-defined minimum.

Generating Item Sets

F	S	I	Risk
Yes	Yes	No	High
Yes	No	No	High
No	No	Yes	Medium
Yes	Yes	Yes	Low
Yes	Yes	No	High
No	Yes	No	Low
Yes	No	Yes	High

Minimum coverage = 3

"1-item" item sets:

F = yes;

S = yes; S = no;

I = yes; I = no;

Risk = High

"2-item" item sets:

F = yes, S = yes;

F = yes, I = no;

F = yes, Risk = High;

I = no, Risk = High;

"3-item" item sets:

F = yes, I = no, Risk = High;

Rule Generation

- First find all combinations of attribute-value pairs with a pre-specified minimum coverage.
- These are called *item-sets*.
- Next Generate all possible rules from the item sets;
- Compute the coverage and accuracy of each rule.
- Prune away rules with accuracy below pre-defined minimum.

Example rules generated

F	S	I	Risk
Yes	Yes	No	High
Yes	No	No	High
No	No	Yes	Medium
Yes	Yes	Yes	Low
Yes	Yes	No	High
No	Yes	No	Low
Yes	No	Yes	High

Minimum coverage = 3

Rules from F= yes:

IF _ then F= yes;
(coverage 5, accuracy 5/7)

Example Rules Generated

F	S	I	Risk
Yes	Yes	No	High
Yes	No	No	High
No	No	Yes	Medium
Yes	Yes	Yes	Low
Yes	Yes	No	High
No	Yes	No	Low
Yes	No	Yes	High

Minimum coverage = 3

Rules from F= yes, S=yes:

IF S = yes then F= yes;
(coverage 3, accuracy 3/4)

IF F = yes then S = yes
(coverage 3, accuracy 3/5)

IF _ then F=yes and S=yes
(coverage 3, accuracy 3/7)

Example rules generated

F	S	I	Risk
Yes	Yes	No	High
Yes	No	No	High
No	No	Yes	Medium
Yes	Yes	Yes	Low
Yes	Yes	No	High
No	Yes	No	Low
Yes	No	Yes	High

Minimum coverage = 3

Rules from : F= yes, I = no, Risk = High;

IF F=yes and I=no then Risk=High (3/3)

IF F=yes and Risk=High then I=no (3/4)

IF I=no and Risk=High then F=yes (3/3)

IF F=yes then I=no and Risk=High (3/5)

IF I=no then Risk=High and F=yes (3/4)

IF Risk=High then I=no and F=yes (3/4)

IF _ then Risk=High and I=no and F=yes
(3/7)

Rule Generation

- First find all combinations of attribute-value pairs with a pre-specified minimum coverage.
- These are called *item-sets*.
- Next Generate all possible rules from the item sets;
- Compute the coverage and accuracy of each rule.
- Prune away rules with accuracy below pre-defined minimum (eg 100%)

Example rules generated

F	S	I	Risk
Yes	Yes	No	High
Yes	No	No	High
No	No	Yes	Medium
Yes	Yes	Yes	Low
Yes	Yes	No	High
No	Yes	No	Low
Yes	No	Yes	High

Minimum coverage = 3

Rules from : F= yes, I = no, Risk = High;

IF F=yes and I=no then Risk=High (3/3)

~~IF F=yes and Risk=High then I=no (3/4)~~

IF I=no and Risk=High then F=yes (3/3)

~~IF F=yes then I=no and Risk=High (3/5)~~

~~IF I=no then Risk=High and F=yes (3/4)~~

~~IF Risk=High then I=no and F=yes (3/4)~~

~~IF _ then Risk=High and I=no and F=yes _____
(3/7)~~

If we require 100% accuracy...

- Only two rules qualify:
- IF I=no and Risk=High then F=yes

IF F=yes and I=no then Risk=High

(Note: second happens to be a rule that has the class attribute on the rhs, in general this need not be the case).

Classifiers v Clustering

Classifiers assign instances to **pre-defined** class labels

But what if training data is not labelled with class? –
unsupervised ML

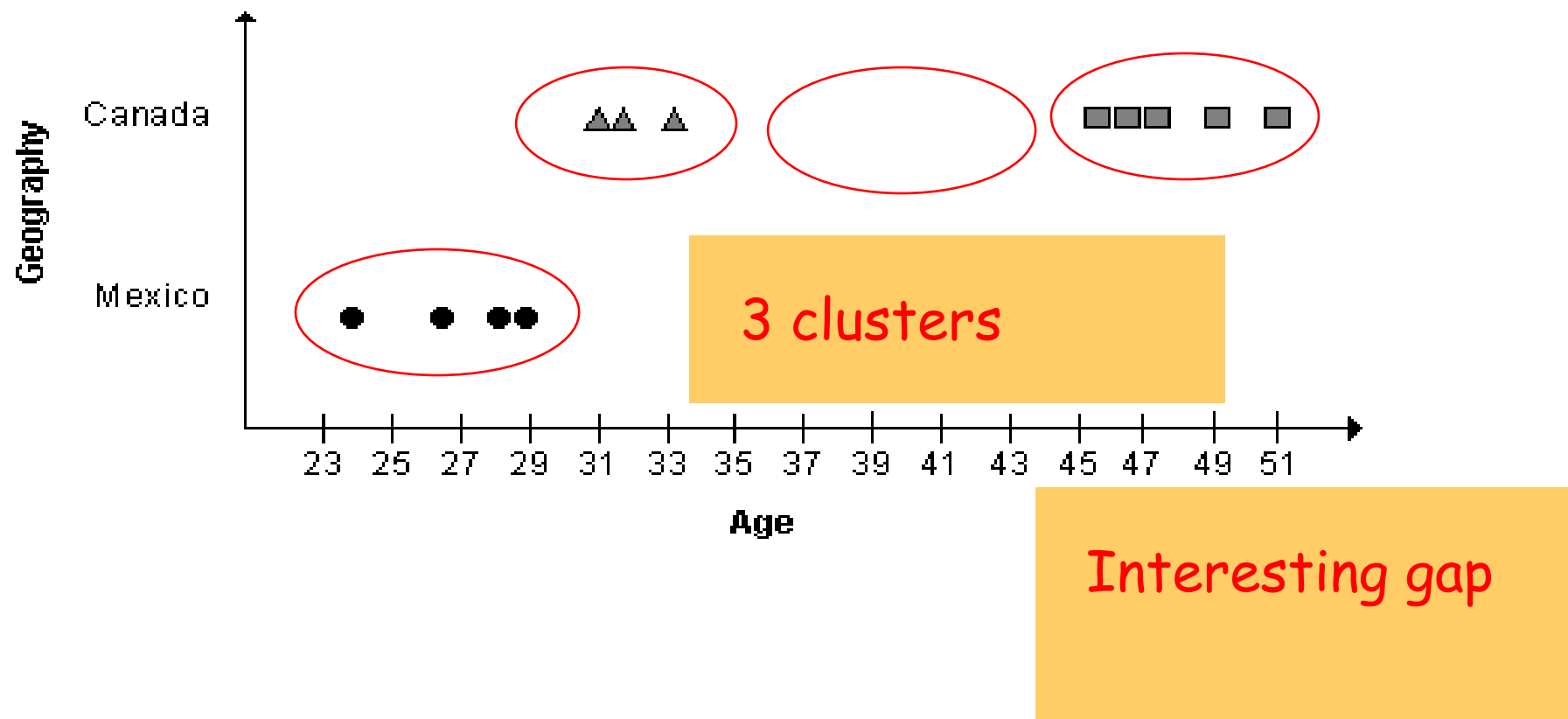
Association rules don't group instances into classes, but
find links between any features / attributes

- Clustering is for discovering 'natural' groups (clusters) which arise from the raw (unlabelled) data.
- Analysis of clusters may lead to knowledge regarding underlying mechanism for their formation.

Example: What clusters can you see?

Customer age	Country travelled to
23	Mexico
45	Canada
32	Canada
47	Canada
46	Canada
34	Canada
51	Canada
28	Mexico
49	Canada
29	Mexico
26	Mexico
31	Canada

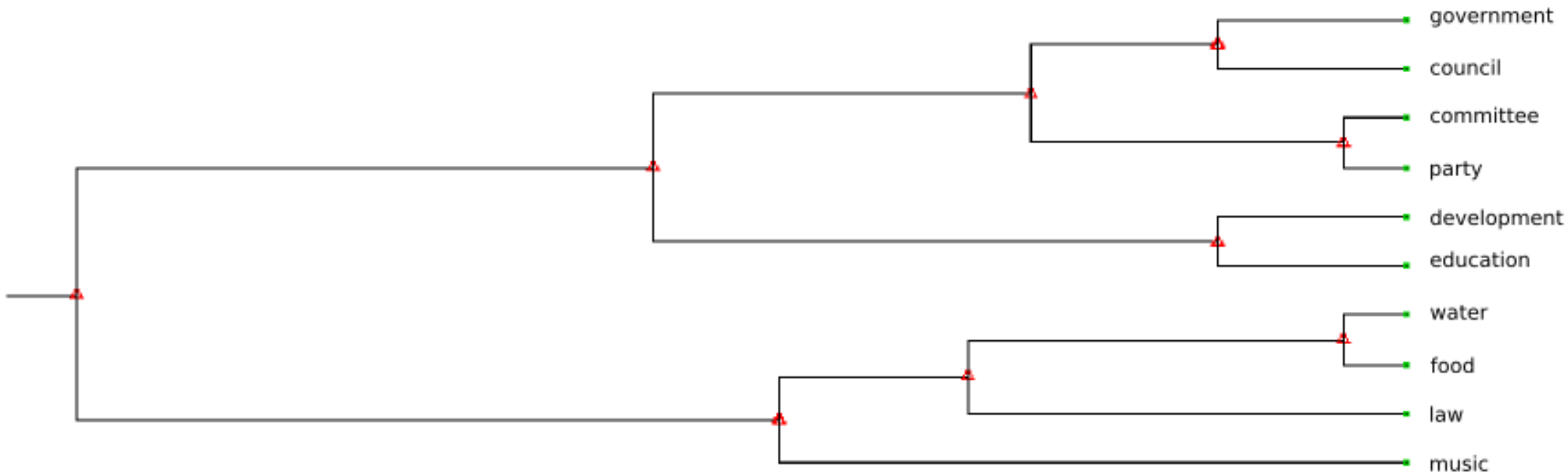
Example



You can try to “explain” the clusters

- Young folk are looking for excitement perhaps, somewhere their parents haven't visited?
- Older folk visit Canada more, Why?
- Particularly interesting is the gap. Probably the age where they can't afford expensive holidays and educate the children
- The client (domain expert – eg travel agent) may “explain” clusters better, once shown them

Hierarchical Clustering: dendrogram



An example of clustering output
visualised as a dendrogram

N-dimensional data

- Consider point of sale data:

- item purchased

- price

- profit margin

- promotion

- store

- shelf-length

- position in store

- date/time

- customer postcode

Some of these are numeric attributes:

(price, profit margin, shelf-length, date-time);

some are nominal:

(item purchased, store, position in store, customer postcode)

To cluster, we need a Distance function

- For some clustering methods (eg K-means) we need to define the distance between two facts, using their vectors.
- Euclidean distance is usually fine:

$$D(\underline{v}, \underline{v}') = \sqrt{\sum_i (v_i - v_i')^2}$$

- Although we usually have to normalise the vector *components* to get good results

Vector Representation

- Represent each instance (fact) as a vector:
 - one dimension for each numeric attribute
 - some nominal attributes may be replaced by numeric attributes (eg postcode to 2 grid coordinates)
 - some nominal attributes replaced by N binary dimensions
 - one for each value that the attribute can take. (eg ‘female’ becomes $\langle 1, 0 \rangle$, ‘male’ becomes $\langle 0, 1 \rangle$)

Example vector:

(0,0,0,0,1,0,0,4.65,15,0,0,1,0,0,0,0,1,....)

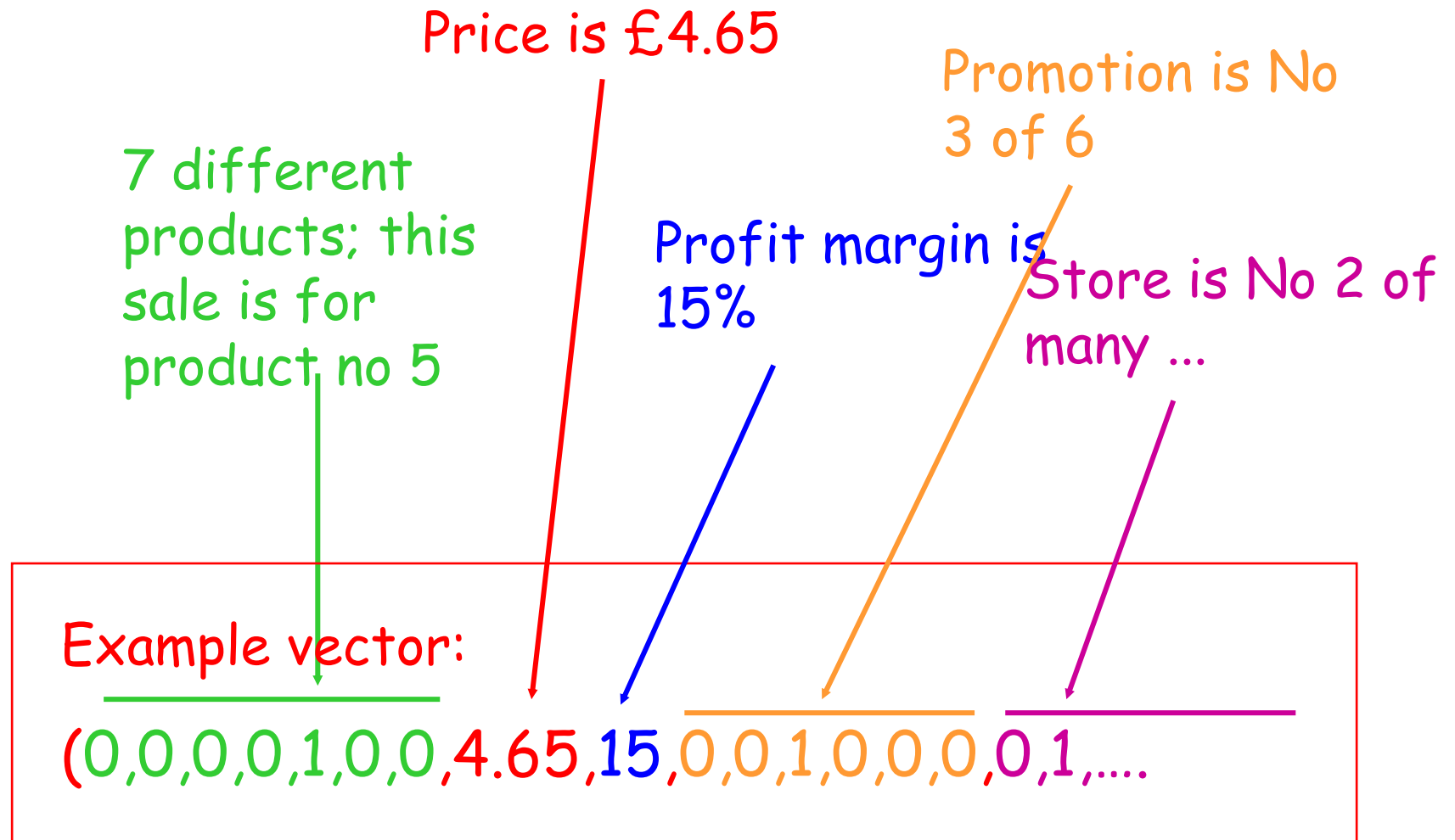
Vector Representation

- Represent each fact as a vector:
 - one dimension for each numerical attribute
 - some nominal attributes are converted to numeric attributes (eg position)
 - some nominal attributes are converted to binary attributes
 - one for each value
- Treatment of nominal features is just like a line in ARFF file; or keyword weights that index documents in IR e.g. Google

Example vector:

(0,0,0,0,1,0,0,4.65,15,0,0,1,0,0,0,0,1)

Vector Representation



Cluster Algorithm

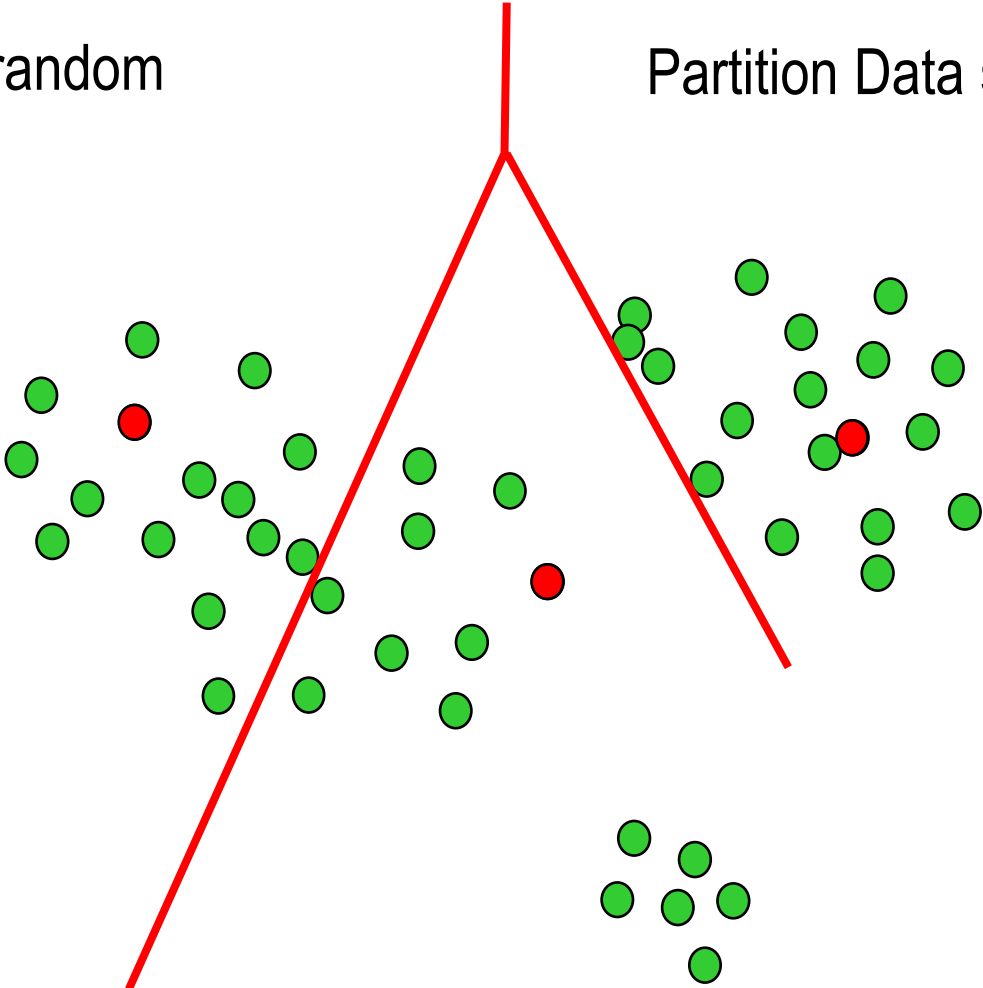
- Now we run an algorithm to identify clusters: n -dimensional regions where facts are dense.
- There are very many cluster algorithms, each suitable for different circumstances.
- We briefly describe k -means iterative optimisation, which yields K clusters; then an alternative incremental method which yields a dendrogram or hierarchy of clusters

Algorithm1: K-means

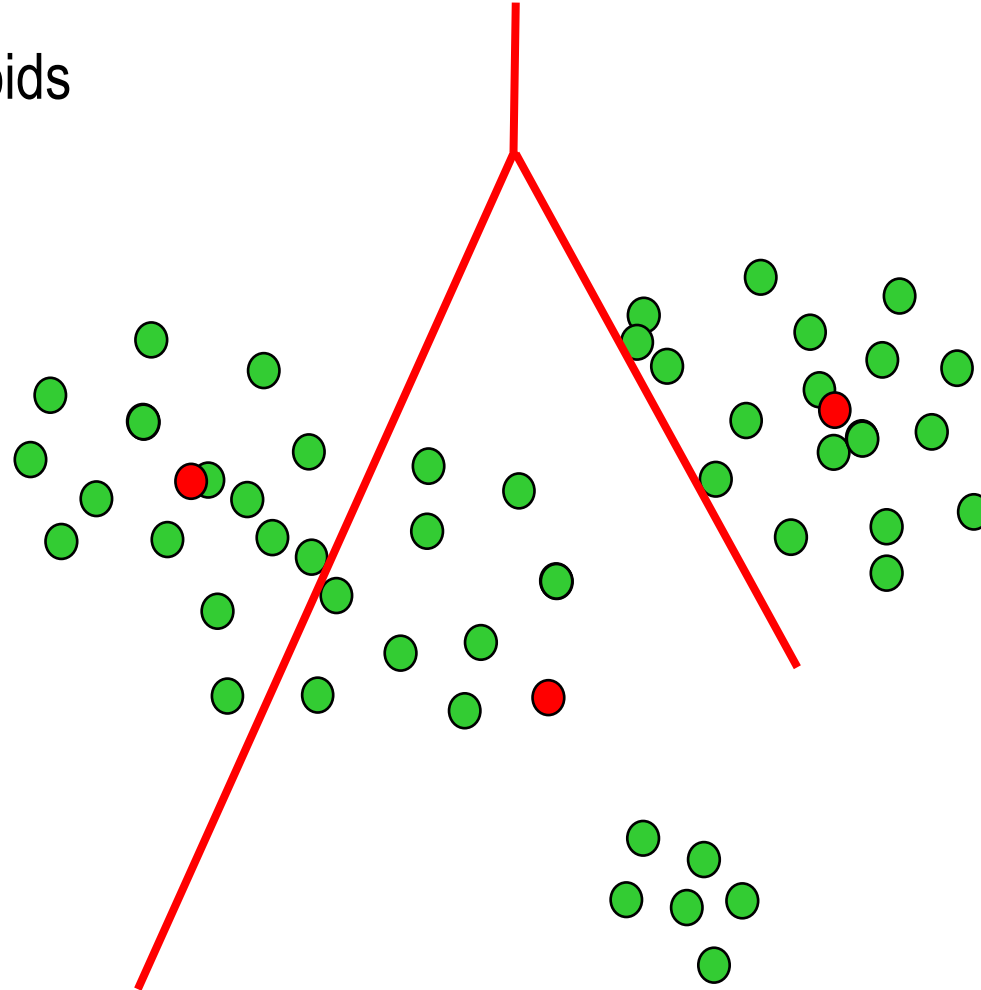
1. **Decide on the number, k , of clusters you want**
 2. Select at random k vectors
 3. Using the distance function, form groups by assigning each remaining vector to the nearest of the k vectors from step 2.
 4. Compute the centroid (mean) of each of the k groups from 3.
 5. Re-form the groups by assigning each vector to the nearest centroid from 4.
 6. Repeat steps 4 and 5 until the groups no longer change.
- The k groups so formed are the clusters.

Pick three points at random

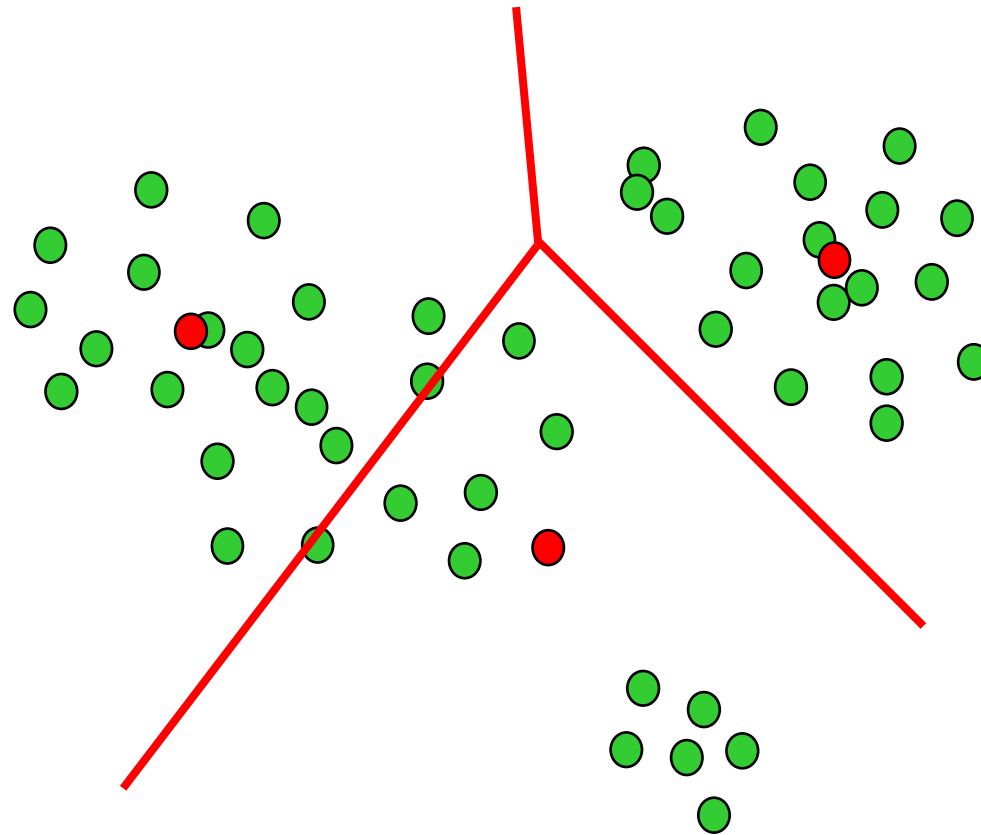
Partition Data set



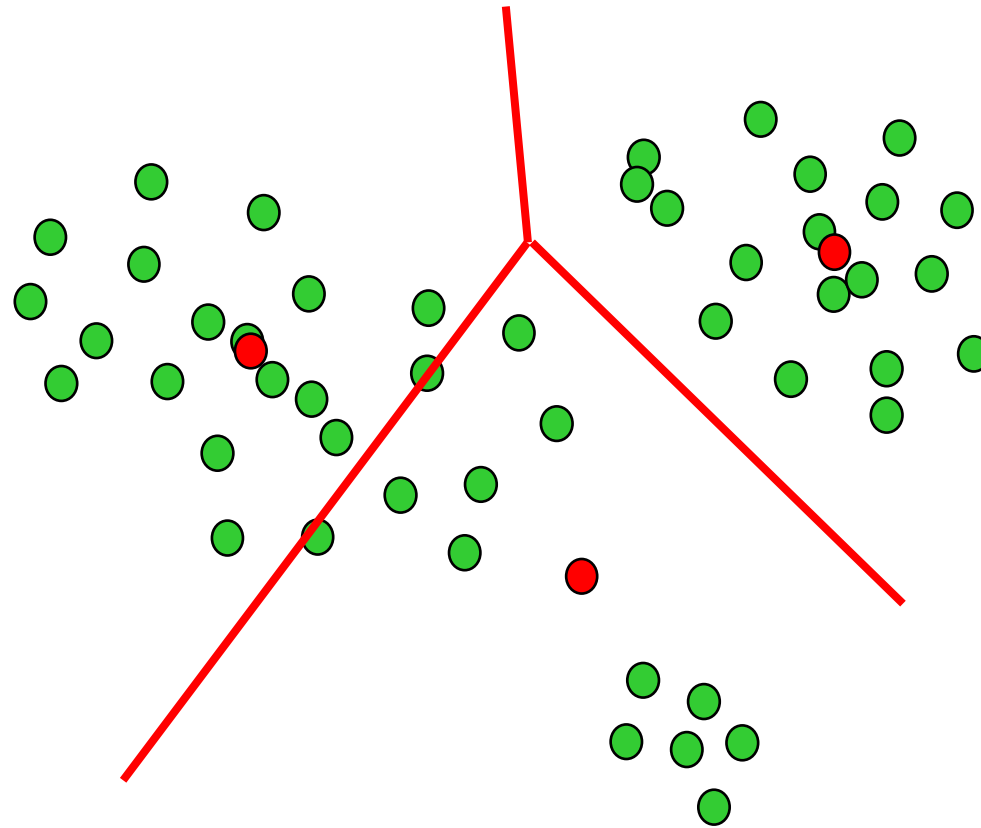
Find partition centroids



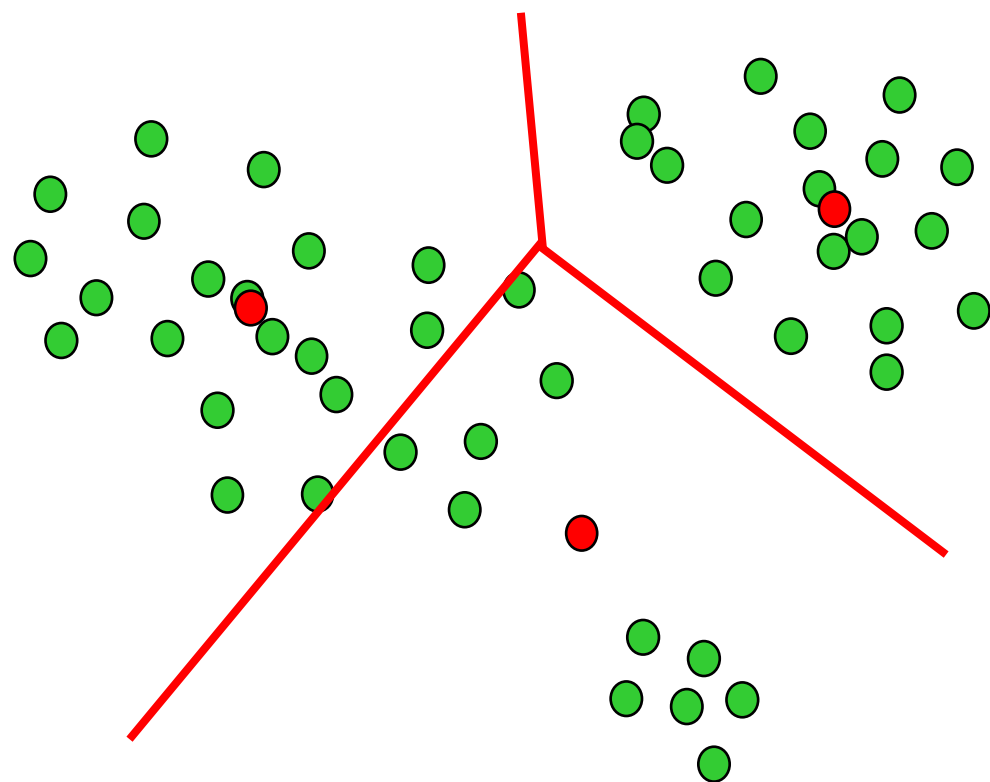
Re-partition



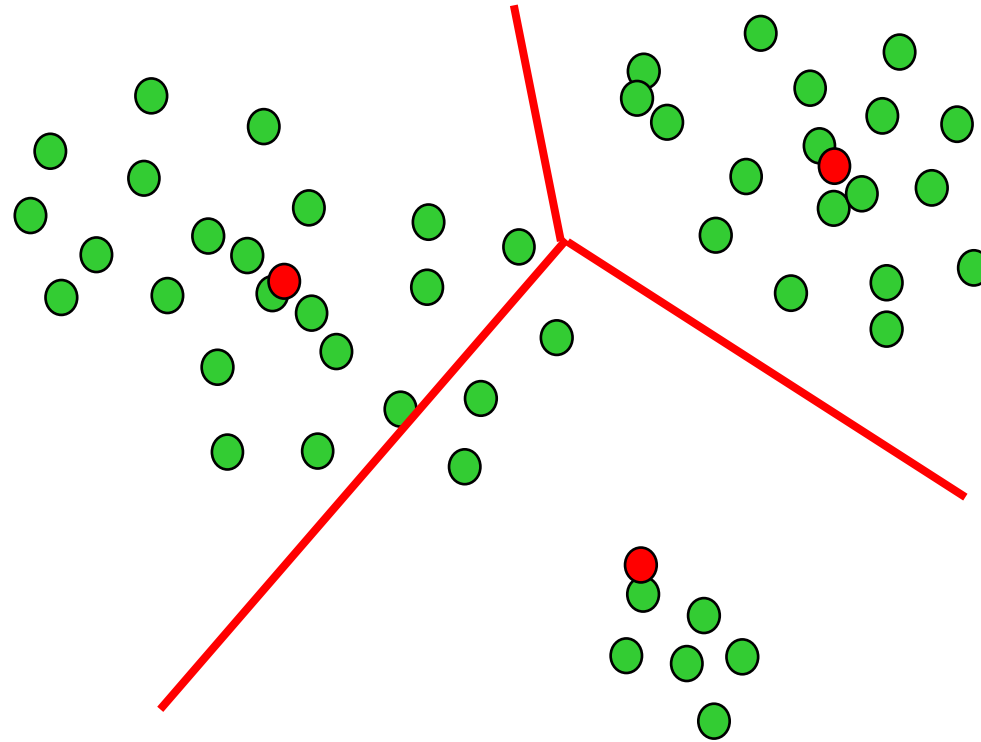
Re-adjust centroids



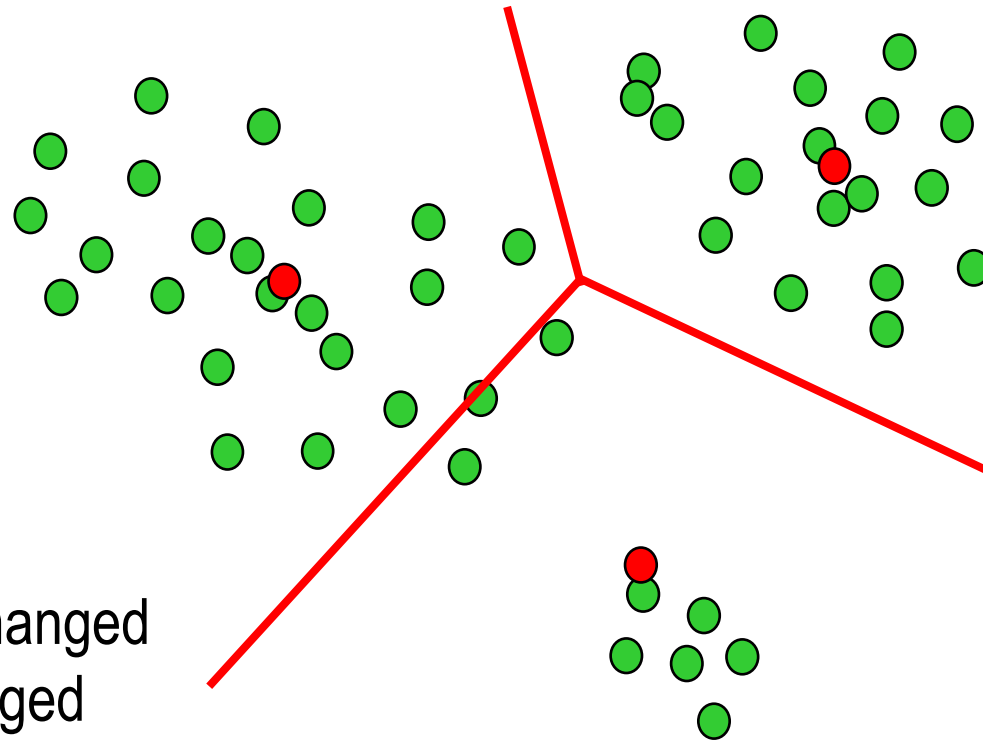
Repartition



Re-adjust centroids



Repartition

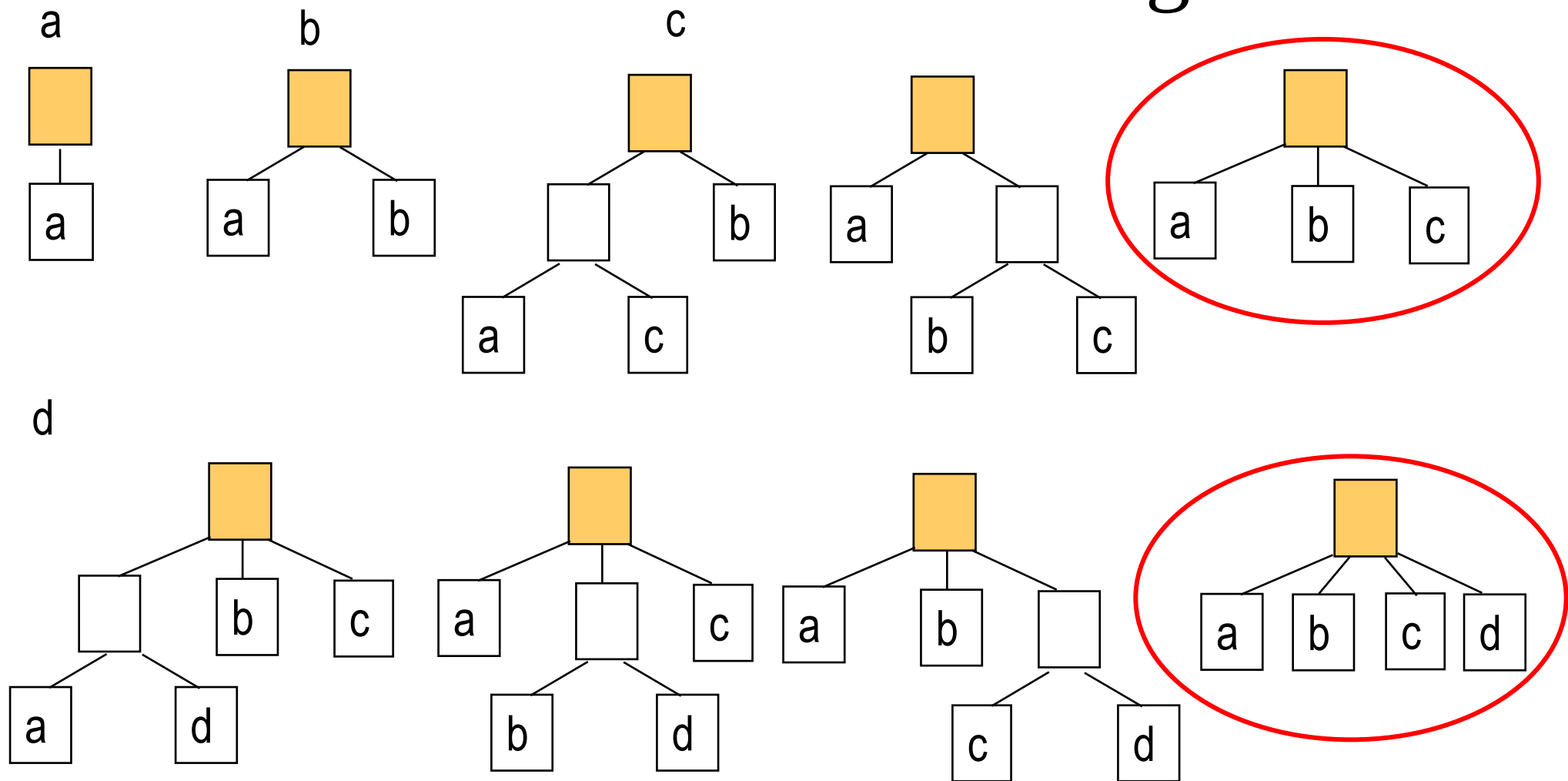


Clusters have not changed
k-means has converged

Algorithm2: Incremental Clustering

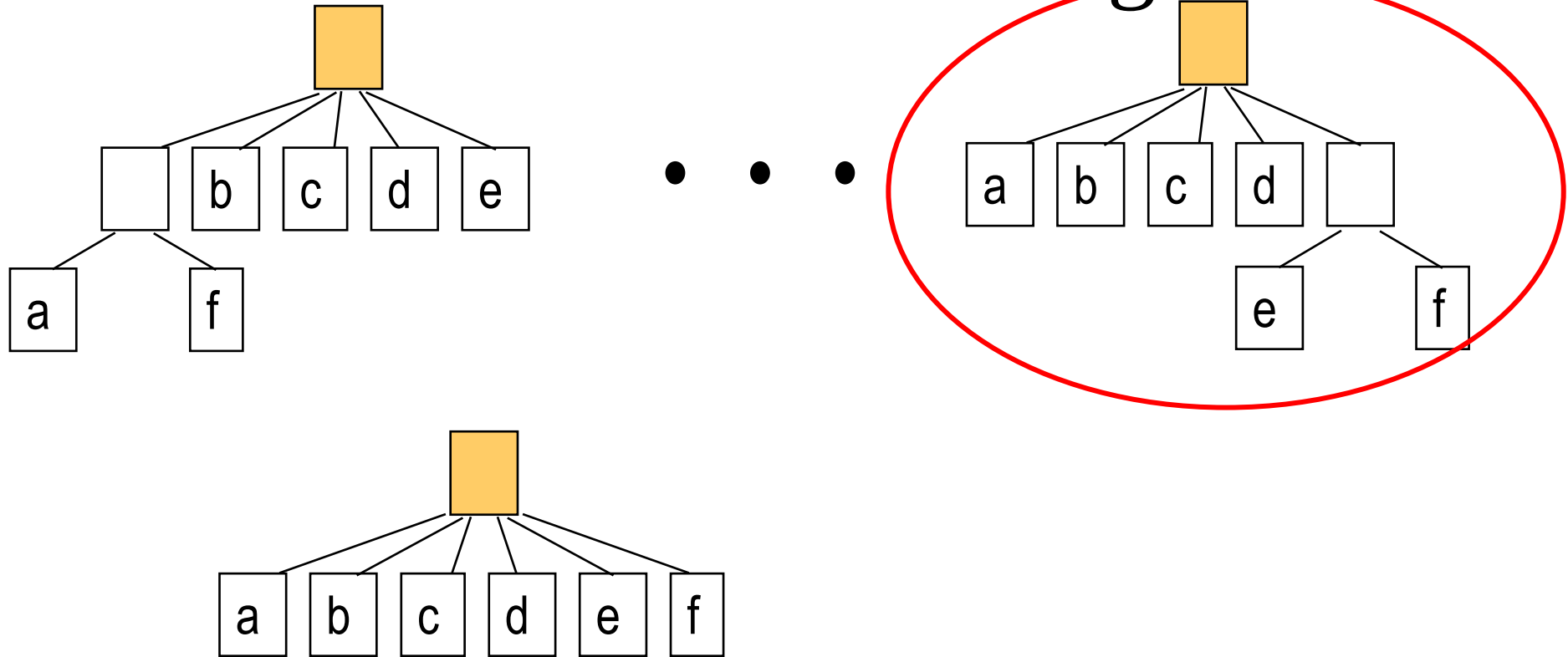
- This method builds a dendrogram “tree of clusters” by adding one instance at a time.
- The decision as to which cluster each new instance should join (or whether they should form a new cluster by themselves), is based on a **category utility**
- The category utility is a measure of how good a particular partition is; it does not require attributes to be numeric.
- Algorithm: for each instance, add to tree so far, where it “best fits” according to category utility

Incremental Clustering



Incremental Clustering

f



Incremental Clustering v K-Means

- Neither method guarantees a globally optimised partition.
- K-means depends on the number of clusters as well as initial seeds (K first guesses).
- Incremental clustering generates a hierarchical structure that can be examined and reasoned about.
- Incremental clustering depends on the order in which instances are added.

Key concepts

Classifiers assign instances to **pre-defined** class labels – supervised ML

But what if instances in training data are not labelled with class? – unsupervised ML

Association rules don't group instances into classes, but find links between any features / attributes

Clustering is for discovering 'natural' groups (clusters) which arise from the raw (unlabelled) data.

Review

- compare classification with **association rules**.
- an algorithm for mining association rules.
- compare classification with **clustering**.
- algorithms for clustering
- the difference between Supervised ML (classifiers) and Unsupervised ML (association, clustering)

Self Check

- Explain the difference between classification and association rules.
- Explain the difference between classification and clustering
- Explain what is meant by cluster centroid, k-means, unsupervised machine learning.